

Depuración en Sistemas Empotrados ANIoT

Carlos García Sánchez

1 de octubre de 2025

- “OpenOCD User Guide”,
<https://openocd.org/doc/pdf/openocd.pdf>

Outline

1 Bare Metal

2 JTAG

3 OpenOCD

Escenarios habituales

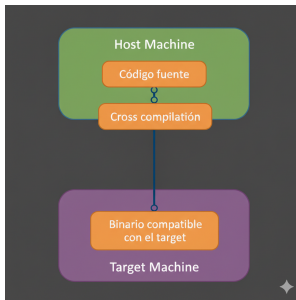
- Para Sistemas Empotrados nos encontramos habitualmente con 3 escenarios diferentes:
 - Desarrollo con SO de propósito general
 - Desarrollo con SO de tiempo real (o firmware)
 - Desarrollo sin SO o *Bare-Metal*

Desarrollo sin sistema operativo

- Este escenario es frecuente para sistemas muy simples
 - Tenemos todo el control del hardware
 - No tenemos sobrecarga de ningún tipo
 - No tenemos servicios, tenemos que programarlo todo nosotros o usar bibliotecas desarrolladas por otros
 - Desarrollamos en equipo host
 - Sólo podemos depurar en circuito si el hardware incorpora facilidad de depuración, por ejemplo JTAG

Compilación Cruzada

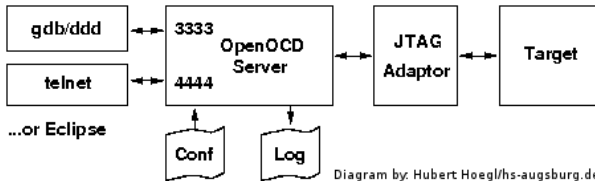
- Toolchain de compilación: permiten binarios compatibles con arquitectura “target”
- El binario no se construyó en plataforma donde se construye



Depuración

- Inspecciona el contenido de la memoria FLASH y RAM (registros)
- Detiene el programa (*breakpoints*)
- Ejecución paso a paso
- Modifica variables en tiempo de ejecución

JTAG



- **Target:** sistema empotrado
- **JTAG Adaptor:** se conecta al puerto JTAG del sistema empotrado y al host por un puerto estándar (USB, PCIE, ...)
- **OpenOCD:** es un programa que se ejecuta en el host, y que sabe comunicarse a través del adaptador JTAG con el HW de depuración en circuito.
- **GDB/DDD:** cliente de depuración, conectado al puerto GDB Server abierto por OpenOCD para controlar la depuración
 - Puede ser un cliente gráfico como Eclipse/DDD o VS-Code

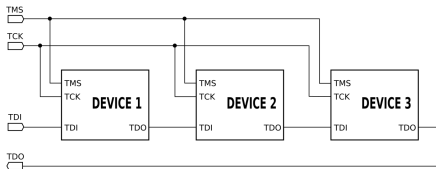
JTAG

- Estándar IEEE 1149.1
 - *IEEE Standard Test Access Port and Boundary-Scan Architecture*
 - Creado en 1990 en una acción conjunta por estandarizar el hardware que permitiese testar los productos electrónicos cada vez más pequeños
 - Actualmente utilizado con varios propósitos:
 - Test de placas (Boundary Scan)
 - Programación de flash
 - Depuración en circuito

Interfaz JTAG

- Uso en Debugging: su principal función en sistemas empotrados es proporcionar un canal de comunicación de bajo nivel al corazón (core) del procesador (CPU) y a los periféricos, incluso cuando el sistema está fallando o detenido.
- Permite depuración no intrusiva

Interfaz JTAG



- Versión de 5 pines:
 - TDI (Test Data In)
 - TDO (Test Data Out)
 - TCK (Test Clock)
 - TMS (Test Mode Select)
 - TRST (Test Reset, opcional)
- Conexión Daisy-Chain de TDI/TDO al estilo SPI
- Los datos se transfieren bit a bit a través de un registro de desplazamiento que conecta en serie a todos los chips

Introducción a OpenOCD

Open On-Chip Debugger (OpenOCD)

Herramienta de depuración y programación de código abierto.

- **Objetivo Principal:** Proporcionar depuración (debugging), programación en sistema (in-system programming) y pruebas de *boundary-scan* para dispositivos *embedded*.
- **Naturaleza:** Es un proyecto de código abierto activo, impulsado por una comunidad diversa de desarrolladores de software y hardware desde su creación en 2005.
- Dispositivos soportados y [más info](#)

Requisito Clave: El Adaptador de Depuración

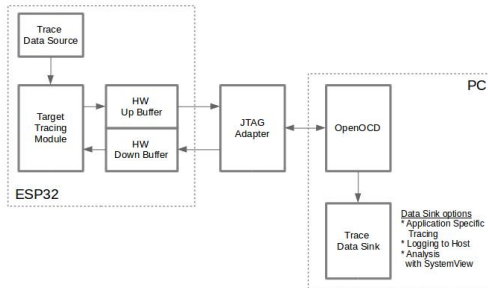
Conexión del Host al Target

OpenOCD requiere un **adaptador de depuración (dongle)** para funcionar.

- **Función:** Proporciona la señalización eléctrica correcta para comunicarse con el dispositivo de destino (target).
- **Protocolos Soportados:** OpenOCD se enfoca principalmente en **JTAG (IEEE 1149.1)**, pero también soporta **SWD (Serial Wire Debug)** y otros protocolos.
- **Tipos de Adaptadores Comunes:** Basados en USB FT2232, USB J-Link o adaptadores basados en **ST-LINK** y CMSIS-DAP.

Conectividad a JTAG

- JTAG: traduce las señales entre el ESP32 y el Host PC (USB/Serial a JTAG)



Configuración y Ejecución

El Servidor y los Archivos de Configuración

OpenOCD se ejecuta como un **servidor** que espera conexiones de clientes (GDB, Telnet o RPC).

- **Archivos de Configuración (.cfg):** Utiliza comandos basados en el lenguaje de *scripting* **Jim Tcl**.
 - `interface/*.cfg`: Define el adaptador de depuración (dongle).
 - `board/*.cfg`: Define la placa y la inicialización de hardware externo (ej. SDRAM, Flash).
 - `target/*.cfg`: Define los *Test Access Ports* (TAP) y las CPUs.
- **Uso Básico:** `openocd -f interface/ADAPTER.cfg -f board/MYBOARD.cfg`.

Integración con GDB

La Columna Vertebral de la Depuración

OpenOCD actúa como un servidor *gdbserver* remoto.

- **Protocolo:** Cumple con el protocolo **GDB remoto**.
- **Conexión:** GDB se conecta a OpenOCD a través de un *socket* TCP/IP (típicamente `localhost:3333`) usando `target extended-remote`.
- **Soporte RTOS:** Puede configurarse para detectar y trabajar con estructuras de datos de RTOS conocidos como **FreeRTOS**, **eCos**, **ThreadX** y **Linux**.

Caso de uso

■ Lanzar servidor OpenOCD

```
user@host:~$ openocd -f board/esp32c3-builtin.cfg
Open On-Chip Debugger v0.12.0-esp32-20250707 (2025-07-06-17:37)
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
Info : esp_usb_jtag: VID set to 0x303a and PID to 0x1001
Info : esp_usb_jtag: capabilities descriptor set to 0x2000
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
....
```


Caso de uso

- Telnet al puerto 4444 que permite interactuar empleando los comando TCL
 - **targets**: muestra los targets (cores) detectados
 - **reset**: reinicia el ESP32 y los detiene (halt)
 - **resume**: reanuda la ejecución

```
user@host:~$ telnet localhost 4444
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Open On-Chip Debugger
> targets

```

	TargetName	Type	Endian	TapName	State
--	-----	-----	-----	-----	-----
0*	esp32c3	esp32c3	little	esp32c3.tap0	running
....					

Caso de uso

- Depuración a través del puerto 3333 con **GDB**
 - Tener en cuenta el entorno de compiación cruzada
`riscv32-esp-elf-gdb -ex "target remote :3333" firmware.elf`
- Pasos a seguir:
 - 1 Conectarse al servidor OpenOCD (puerto 3333)
 - 2 Reiniciar el micro-controlador: `mon reset halt`
 - 3 Punto de ruptura, ej: inicio de la función: `thb app_main`
 - 4 c continuar, n next, s step, p imprimir variable

```
user@host:~$ riscv32-esp-elf-gdb -ex "target remote :3333" build/Pi-blink.elf
(gdb) mon reset halt
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif
Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) thb app_main
Hardware assisted breakpoint 1 at 0x42004d96: file
Pi-blink/main/blink_example_main.c, line 95.
....
```

Caso de uso

- Integración con el VS-Code. Pasos a seguir
 - 1 Select Flash Method **JTAG**
 - 2 Lanzar el servidor OpenOCD `openocd -f board/esp32c3-builtin.cfg`
 - 3 En el entorno del VS-Code clicar el icono **Debug** o **F5**
 - 4 Se puede monitorizar variables, colocar breakpoints, o consultar variables

Caso de uso

■ Integración con el VS-Code

